



Assignment Code: DA-AG-004

Restful API & Flask | Assignment

Instructions: Carefully read each question. Use Google Docs, Microsoft Word, or a similar tool to create a document where you type out each question along with its answer. Save the document as a PDF, and then upload it to the LMS. Please do not zip or archive the files before uploading them. Each question carries 20 marks.

Total Marks: 200

Question 1 : What is a RESTful API?

Answer:

A RESTful API is an Application Programming Interface (API) that follows the principles of Representational State Transfer (REST), an architectural style for designing networked applications. RESTful APIs use standard web technologies to facilitate secure and efficient communication between a client (like a web browser or mobile app) and a server.

Key principles of REST

For an API to be considered truly "RESTful," it must adhere to a set of constraints:

- **Client-Server:** The client (e.g., your web browser) and the server are separate entities that can evolve independently. The client is responsible for the user interface, while the server stores and processes the data.
- **Stateless:** The server does not store any information about the client's session between requests. Each request from the client must contain all the information the server needs to fulfill it.
- **Cacheable:** Responses from the server must be defined as either cacheable or non-cacheable. This helps reduce network traffic and improves performance for the user.
- **Uniform Interface:** There is a standard and uniform way of interacting with the API, regardless of the client or the specific device being used.
- **Layered System:** A client cannot tell if it is connected directly to the end server or to an intermediary, such as a load balancer or a proxy. This allows for improved scalability and security.

Example of a RESTful request:

Request: GET <https://api.example.com/users/1234>

Response: The server returns an HTTP status code 200 OK along with a JSON body containing the user's data.

Question 2: What is Flask, and why is it popular for building APIs?

Answer:

Introduction

A RESTful API (Application Programming Interface) is a standardized architectural style for building network-based software applications. Instead of being a specific technology, REST sets a collection of rules and constraints for how a client and server can communicate. [Flask](#), a lightweight Python web framework, has become an exceptionally popular tool for developing these APIs due to its simplicity, flexibility, and extensibility. This answer explores the core principles of REST, demonstrates how Flask facilitates the construction of RESTful APIs, and explains why this combination is favored by many developers.

The principles of RESTful APIs

For an API to be considered truly RESTful, it must adhere to several key architectural principles:

- **Client-Server:** The client and server are distinct entities that can evolve independently. The client handles the user interface, while the server stores and processes the data.
- **Stateless:** Each request from the client to the server must contain all the information the server needs to process it. The server does not store any session information, which improves scalability and reliability.
- **Cacheable:** The server's responses must explicitly state whether they can be cached. This helps reduce client-server interactions, improving performance and efficiency.
- **Uniform Interface:** This principle is fundamental to REST. It simplifies and decouples the architecture by requiring that all interactions with resources follow a uniform, predictable pattern, typically using standard HTTP methods.
- **Layered System:** A client may not know if it is communicating directly with the end server or with an intermediary like a load balancer. This tiered approach enhances scalability and security.
- **Code-on-Demand (optional):** The server can send executable code to the client to extend its functionality, though this is not a mandatory constraint for a RESTful API.

Common RESTful operations and HTTP methods

RESTful APIs leverage standard HTTP methods to perform operations, often corresponding to the basic CRUD (Create, Read, Update, Delete) functions:

- **GET:** Retrieves a resource or a collection of resources.
- **POST:** Creates a new resource.
- **PUT:** Replaces an existing resource with new data.
- **PATCH:** Applies a partial update to a resource.
- **DELETE:** Deletes a resource.

Question 3: What are HTTP methods used in RESTful APIs?

Answer:

HTTP methods, also known as verbs, are a fundamental part of the HTTP protocol that RESTful APIs use to perform specific actions on resources. They correspond to the basic CRUD (Create, Read, Update, Delete) operations, allowing clients to manipulate resources on a server in a standardized and predictable way.

The most common HTTP methods used in RESTful APIs are:

GET

- **Purpose:** To retrieve a representation of a resource or a collection of resources.
- **Safety and Idempotence:** It is considered a "safe" method because it does not alter the state of the server. It is also "idempotent," meaning that making the same request multiple times will have the exact same outcome as a single request.
- **Usage:** A GET request to /users retrieves a list of all users, while a GET request to /users/123 retrieves the specific user with ID 123.

POST

- **Purpose:** To create a new resource on the server.
- **Safety and Idempotence:** It is neither safe nor idempotent. Repeated identical POST requests can create multiple resources and cause different outcomes.
- **Usage:** A POST request is sent to a collection's endpoint (e.g., /users) with the new resource data in the request body. The server creates the new resource and assigns it a unique ID.

PUT

- **Purpose:** To update an existing resource or create a new one at a specific URI.
- **Safety and Idempotence:** It is idempotent, meaning that if you send the same PUT request multiple times, the final state of the resource on the server will be the same as if you had sent it only once. Unlike PATCH, PUT requires a full, complete representation of the resource in the request body.
- **Usage:** A PUT request is sent to a specific resource's endpoint (e.g., /users/123) with the full updated resource data.

PATCH

- **Purpose:** To apply a partial modification to a resource.
- **Safety and Idempotence:** It is not necessarily idempotent, as a PATCH request could, for instance, increment a counter on a resource every time it's called.
- **Usage:** PATCH is used when you only need to update specific fields of a resource instead of replacing the entire resource, making it more efficient for partial updates.

DELETE

- **Purpose:** To delete a specific resource identified by a URI.
- **Safety and Idempotence:** It is considered idempotent. Deleting a resource and then attempting to delete it again will leave the server in the same final state (the resource is gone), though the second request will typically return a 404 Not Found status.

- **Usage:** A DELETE request is sent to a specific resource's endpoint (e.g., /users/123).

Question 4: What is the purpose of the `@app.route()` decorator in Flask?

Answer:

The `@app.route()` decorator in [Flask](#) is used to bind a URL to a Python function. It provides a straightforward way to map incoming web requests to the specific code that handles them. When a user navigates to a URL that matches the one defined in the decorator, Flask executes the decorated function and returns its output as the response.

Core purposes of the `@app.route()` decorator

1. **URL Mapping:** It is the primary tool for mapping a specific URL path to a *view function* (a function that handles requests for a specific URL). For example, `@app.route('/')` binds the root URL of your application to the function that follows it, making that function the handler for the home page.
2. **Defining API Endpoints:** When building a RESTful API with Flask, the decorator is used to define the endpoints for your resources. You define a path (e.g., /users, /posts/<int:post_id>) and a Python function that executes the logic for that endpoint.
3. **Handling Different HTTP Methods:** You can specify which HTTP methods (like GET, POST, PUT, DELETE) a route should respond to by passing a `methods` argument to the decorator. This is essential for building RESTful APIs where different actions are performed based on the HTTP method used.
4. **Dynamic Routing:** The decorator enables dynamic URL patterns by allowing you to include variable parts in the URL, such as <variable_name>. These variables are then passed as arguments to the decorated function.
Default (string): `@app.route('/users/<username>')` will pass the username as a string.
With converters: `@app.route('/posts/<int:post_id>')` restricts the `post_id` variable to be an integer.

How it works behind the scenes

The `@app.route()` decorator is a piece of syntactic sugar for a more explicit method, `app.add_url_rule()`. When Flask loads your application, the decorator runs and registers the specified URL and its corresponding function inside a routing table. This table is a data structure that Flask uses to look up the correct function to execute when a request for a specific URL comes in.

Question 5: What is the role of Flask-SQLAlchemy?

Answer:

Flask-SQLAlchemy is a Flask extension that simplifies the process of integrating the SQLAlchemy Object-Relational Mapper (ORM) into Flask applications. Because Flask is a "micro-framework" and does not include a database layer by default, Flask-SQLAlchemy handles the boilerplate code required to set up and manage database connections, sessions, and models.

Core roles of Flask-SQLAlchemy

1. Provides an Object-Relational Mapper (ORM)

The central role of Flask-SQLAlchemy is to bring SQLAlchemy's ORM capabilities to your Flask app. An ORM allows you to work with database tables using Python classes, eliminating the need to write raw SQL queries for common operations. This leads to:

- **Intuitive data handling:** Developers can perform CRUD (Create, Read, Update, Delete) operations on database records by manipulating Python objects.
- **Reduced boilerplate:** The extension manages the connection details and session management, so you can focus on your application's business logic.
- **Enhanced security:** The ORM automatically handles data escaping, which helps prevent SQL injection attacks.

2. Simplifies configuration

Flask-SQLAlchemy streamlines database configuration by allowing you to define settings directly within your Flask application's configuration file. You simply set a database URI to connect to various database systems like SQLite, PostgreSQL, or MySQL.

3. Manages database sessions

In a web application, managing database sessions is crucial to prevent resource leaks and ensure data consistency. Flask-SQLAlchemy automatically handles the session lifecycle for each web request, ensuring that database connections are properly opened and closed.

4. Handles database modeling

Flask-SQLAlchemy provides a `db.Model` base class that your application's data models inherit from. Each model class represents a database table, and its attributes represent the table's columns. This declarative syntax makes it easy to define your database schema using standard Python syntax.

5. Seamless integration with Flask

By integrating directly with Flask's application context, Flask-SQLAlchemy ensures that database interactions occur smoothly within the request-response cycle. This makes it easier to work with databases inside your view functions and other parts of your Flask application.

6. Supports database migrations

While Flask-SQLAlchemy doesn't handle migrations on its own, it works seamlessly with migration libraries like Flask-Migrate. This allows you to track changes to your database

schema over time and update it in an organized way, which is especially important for applications in production.

Question 6: How do you create a basic Flask application?

Answer:

Step 1: Install Flask

First, set up a virtual environment and install Flask inside of it. A virtual environment is an isolated space that keeps your project's dependencies separate from other Python projects.

1. **Create a project directory** and navigate into it.
2. **Create a virtual environment** by running:
 - `python3 -m venv venv`
3. **Activate the virtual environment:**
 - **macOS/Linux:** `source venv/bin/activate`
 - **Windows:** `venv\Scripts\activate`
4. **Install Flask** using pip:
 - `pip install Flask`

Step 2: Write the application code

Create a new file, for instance `app.py`, inside your project directory. It is important not to name your file `flask.py` to avoid conflicting with the Flask package itself.

In your `app.py` file, add the following code:

```
from flask import Flask
```

```
# Create an instance of the Flask class
```

```
app = Flask(__name__)
```

```
# Define a route for the home page ('/')
```

```
@app.route('/')
def hello_world():
```

```
    return '<p>Hello, World!</p>'
```

```
# Run the application
```

```
if __name__ == '__main__':
```

```
    app.run(debug=True)
```

- **from flask import Flask:** This line imports the Flask class, which is necessary for creating your application.
- **app = Flask(__name__):** This creates an instance of the Flask class, with `__name__` as the argument. Flask needs this to know where to find resources like templates.
- **@app.route('/')**: This is a decorator that binds a URL path ('/' for the home page) to the function immediately following it.

- **def hello_world():** This is the *view function* that handles requests for the specified route and returns the content to be displayed in the browser.
- **if __name__ == '__main__':** This standard Python construct ensures the development server runs only when you execute the script directly.
- **app.run(debug=True):** This starts the Flask development server. debug=True enables debug mode, which automatically reloads the server on code changes and provides a helpful debugger in the browser

Step 3: Run the application

With the virtual environment activated, run the application from your terminal:

bash

python app.py

Use code with caution.

Alternatively, Flask's command-line interface (CLI) is the recommended way to start the server.

1. Set the FLASK_APP environment variable:
 - **macOS/Linux:** export FLASK_APP=app.py
 - **Windows:** set FLASK_APP=app.py
2. Run the Flask application:
 - **flask run**

Question 7: How do you return JSON responses in Flask?

Answer:

In Flask, you can return JSON responses using the built-in jsonify function or by simply returning a Python dictionary or list. The framework handles the conversion and sets the appropriate Content-Type header (application/json) automatically.

Method 1: Using the jsonify function

The jsonify function is the standard and most explicit way to create a JSON response in Flask. It automatically converts a Python dictionary or list into a Response object with the correct header.

EXAMPLE:

```
from flask import Flask, jsonify
```

```
app = Flask(__name__)
```

```
@app.route('/users/<int:user_id>')
```

```
def get_user(user_id):
```

```
    # In a real application, you would fetch this data from a database
```

```
    users = {
```

```
        1: {'name': 'Alice', 'email': 'alice@example.com'},
```

```
2: {'name': 'Bob', 'email': 'bob@example.com'}  
}
```

```
user = users.get(user_id)  
if user:  
    return jsonify(user) # Returns a JSON object
```

```
return jsonify({'error': 'User not found'}), 404
```

Method 2: Returning a dictionary (recommended for modern Flask)

For Flask versions 1.1.0 and later, you can directly return a Python dictionary or list from a view function, and Flask will automatically convert it to a JSON response.

Example:

```
python  
from flask import Flask  
  
app = Flask(__name__)  
  
@app.route('/users/<int:user_id>')  
def get_user(user_id):  
    users = {  
        1: {'name': 'Alice', 'email': 'alice@example.com'},  
        2: {'name': 'Bob', 'email': 'bob@example.com'}  
    }  
  
    user = users.get(user_id)  
    if user:  
        # Flask will automatically jsonify this dictionary  
        return user  
  
    # Return a dictionary and a status code  
    return {'error': 'User not found'}, 404
```

Question 8: How do you handle POST requests in Flask?

Answer:

Introduction:

In Flask, a POST request is used to **send data from the client to the server**, such as form data or JSON. Flask provides the request object to access the data sent in POST requests. Unlike GET requests, POST requests do not append data to the URL.

Step 1: Import required modules


```
from flask import Flask, request, jsonify
```

- Flask is the framework class.
- request is used to access incoming request data.
- jsonify is used to return a JSON response.

Step 2: Create a Flask application

```
app = Flask(__name__)
```

Step 3: Define a route to handle POST requests

```
@app.route('/submit', methods=['POST'])
```

```
def submit_data():
```

```
    # Access JSON data sent in the request
```

```
    data = request.json
```

```
    # Extract fields
```

```
    name = data.get('name')
```

```
    age = data.get('age')
```

```
    # Return response
```

```
    return jsonify({
```

```
        "message": f"Data received successfully for {name}, age {age}"
```

```
    })
```

Explanation:

- `@app.route('/submit', methods=['POST'])`: Specifies that this route only handles POST requests.
- `request.json`: Accesses the JSON payload sent by the client.
- `data.get('field')`: Extracts individual data fields.
- `jsonify`: Sends a response back to the client in JSON format.

Step 4: Run the Flask app

```
if __name__ == "__main__":
```

```
    app.run(debug=True)
```

Step 5: Sending a POST request

Using **curl**:

```
curl -X POST http://127.0.0.1:5000/submit \
```

```
-H "Content-Type: application/json" \
```

```
-d '{"name": "Alice", "age": 25}'
```

Response:

```
{
  "message": "Data received successfully for Alice, age 25"
}
```

Conclusion:

In Flask, handling POST requests involves defining a route with `methods=['POST']`, accessing the data using `request` (JSON, form, or files), processing it, and returning an appropriate response. This allows secure and structured data submission from client to server.

Question 9: How do you handle errors in Flask (e.g., 404)?

Answer:

Introduction:

In Flask, errors like **404 (Page Not Found)**, **500 (Internal Server Error)**, or **400 (Bad Request)** can occur when a user requests a non-existent page or when the server fails to process a request. Flask provides a built-in mechanism to **catch and handle errors gracefully** using the `@app.errorhandler` decorator. Proper error handling improves user experience and makes debugging easier.

Step 1: Import Flask and required modules

```
from flask import Flask, jsonify, render_template
```

- Flask: The framework to create the web app.
- jsonify: To return JSON responses for errors.
- render_template: To return custom HTML pages (optional).

Step 2: Create a Flask application

```
app = Flask(__name__)
```

Step 3: Define normal routes

```
@app.route('/')
def home():
    return "Welcome to the Home Page"
```

```
@app.route('/about')
def about():
    return "This is the About Page"
```

Step 4: Handle 404 Error (Page Not Found)

```
@app.errorhandler(404)
def page_not_found(error):
    return jsonify({
        "error": "404 Not Found",
```

```
    "message": "The requested URL was not found on the server."
  }}, 404
```

Explanation:

- `@app.errorhandler(404)`: Flask uses this function whenever a 404 error occurs.
- Returns a **JSON response** with an appropriate message and HTTP status code.
- Instead of JSON, you can render a custom HTML page:

```
@app.errorhandler(404)
def page_not_found(error):
    return render_template('404.html'), 404
```

Step 5: Handle other errors (Optional)

```
@app.errorhandler(500)
def internal_server_error(error):
    return jsonify({
        "error": "500 Internal Server Error",
        "message": "An unexpected error occurred on the server."
    }), 500
```

- 500 errors occur due to server-side issues like bugs or exceptions.

Step 6: Global Exception Handling (Optional)

```
@app.errorhandler(Exception)
def handle_exception(e):
    return jsonify({
        "error": "Server Error",
        "message": str(e)
    }), 500
```

- Catches any unhandled exceptions globally.
- Useful for debugging during development.

Step 7: Run the Flask app

```
if __name__ == "__main__":
    app.run(debug=True)
```

Conclusion:

Flask's error handling mechanism allows developers to gracefully manage errors and provide meaningful feedback to the client. Proper implementation of `@app.errorhandler` ensures that users don't see default error pages and that the server can handle exceptions systematically.

Question 10: How do you structure a Flask app using Blueprints?

Answer:

Introduction:

In Flask, a **Blueprint** is a way to organize your application into **modular components**. It allows you to separate routes, templates, and static files into reusable modules, making large applications easier to manage.

Step 1: Import required modules

```
from flask import Flask, Blueprint, jsonify
```

- Flask: To create the app instance.
- Blueprint: To create modular app components.
- jsonify: To send JSON responses.

Step 2: Create a Blueprint

```
# In file: user_routes.py
from flask import Blueprint, jsonify

user_bp = Blueprint('user', __name__, url_prefix='/user')

@user_bp.route('/profile')
def profile():
    return jsonify({"message": "This is the user profile page"})

@user_bp.route('/settings')
def settings():
    return jsonify({"message": "This is the user settings page"})
```

Explanation:

- 'user' → The Blueprint name.
- __name__ → Helps Flask locate resources.
- url_prefix='/user' → All routes in this blueprint will start with /user.

Step 3: Create the main Flask app and register Blueprint

```
# In file: app.py
from flask import Flask
from user_routes import user_bp

app = Flask(__name__)

# Register the blueprint
app.register_blueprint(user_bp)

@app.route('/')
def home():
    return "Welcome to the Home Page"

if __name__ == "__main__":
    app.run(debug=True)
```

Explanation:

- `app.register_blueprint(user_bp)` → Adds all routes from the blueprint to the main app.
- You can create **multiple blueprints** for different modules like admin, blog, auth, etc.

Step 4: Project Folder Structure

A well-structured Flask app with Blueprints:

`my_flask_app/`

```
|
|— app.py          # Main application
|— user_routes.py  # Blueprint for user module
|— admin_routes.py # Blueprint for admin module
|— templates/
|   |— ...         # HTML templates
|— static/
|   |— ...         # CSS, JS, Images
|— __init__.py     # Optional for package structure
```

Step 5: Benefits of Using Blueprints

1. **Modularity** → Each module handles a separate part of the app.
2. **Reusability** → Blueprints can be reused in multiple projects.
3. **Maintainability** → Easier to manage routes, templates, and static files.
4. **Scalability** → Supports large projects without cluttering `app.py`.

Step 6: Example Output

- Accessing `/user/profile` → `{"message": "This is the user profile page"}`
- Accessing `/user/settings` → `{"message": "This is the user settings page"}`
- Accessing `/` → `"Welcome to the Home Page"`

Conclusion:

Using **Blueprints**, Flask apps can be structured in a **modular and organized way**. Each Blueprint encapsulates related routes, templates, and static files, allowing **scalable and maintainable applications**. This is the recommended approach for large Flask projects.